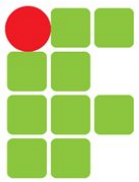


Widgets de Criação de Layouts, Inputs, Botões e Diálogos

Desenvolvimento para Dispositivos Móveis (BRADEMO)

Prof. Luiz Gustavo Diniz de Oliveira Véras

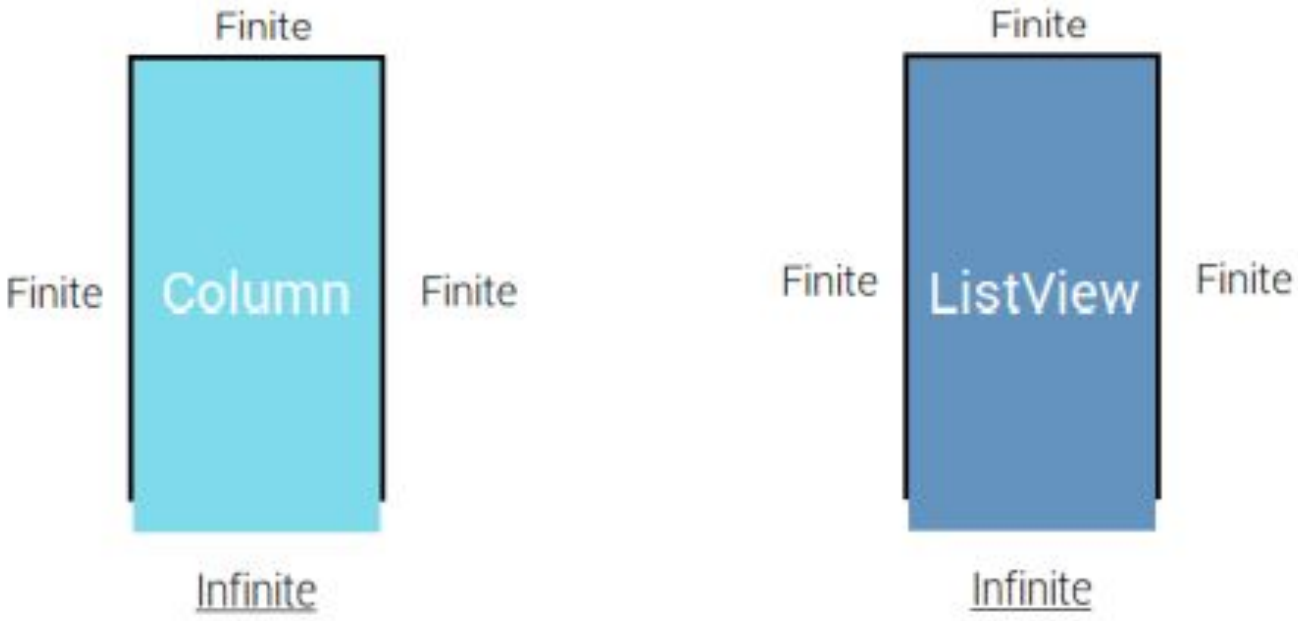
E-mail: gustavo_veras@ifsp.edu.br

Conteúdo

- Scrolling e Constraints
- Widgets Comuns de Layout
- Responsividade
  - *LayoutBuilder*
  - *MediaQuery*
- Grupos de Widgets
  - Input
  - Buttons
  - Dialogs, Popups e Messages

Scrolling e Constraints

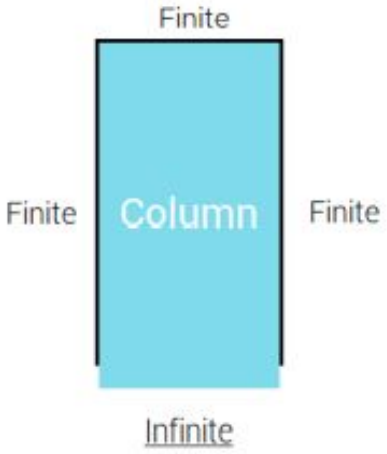
Ambos **ListViews** e **Columns** são muito populares, mas você deve prestar atenção em como eles tratam o conteúdo no eixo vertical.

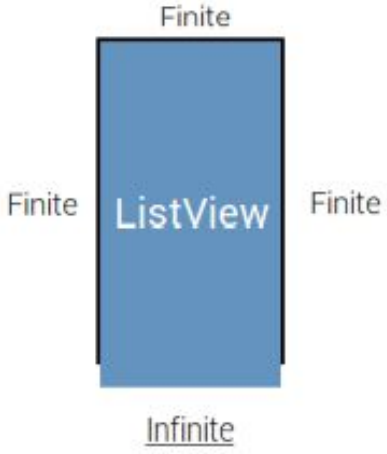


Scrolling e Constraints

Ambos **ListViews** e **Columns** são muito populares, mas você deve prestar atenção em como eles tratam o conteúdo no eixo vertical.

```
Column(
    children: [
        const Text("My name"),
        const Text("My surname"),

        ListView(
            children: const [
                Text("Skill 1"), Text("Skill 2"),
            ]
        )
    ]
);
```



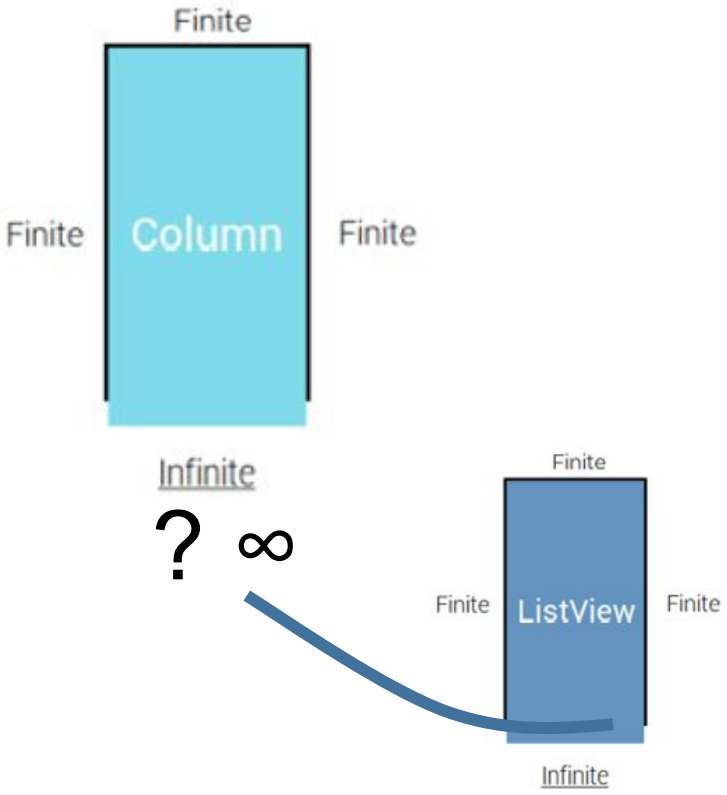


Scrolling e Constraints

Ambos **ListViews** e **Columns** são muito populares, mas você deve prestar atenção em como eles tratam o conteúdo no eixo vertical.

```
Column(
    children: [
        const Text("My name"),
        const Text("My surname"),

        ListView(
            children: const [
                Text("Skill 1"), Text("Skill 2"),
            ]
        )
    ]
);
```

Este código parece funcionar como pretendido, mas lançará um tempo de execução exceção porque a altura da coluna é infinita. (Verificar no DEBUG_CONSOLE)

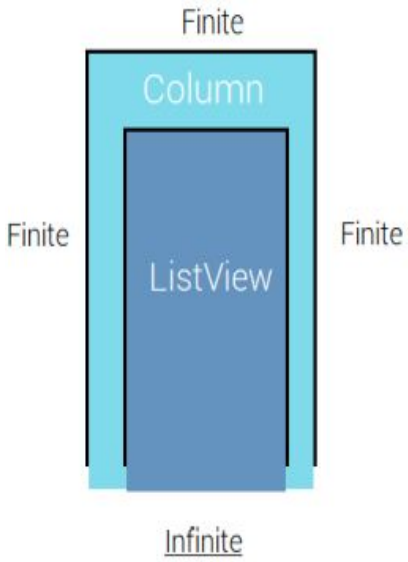


Scrolling e Constraints

A solução acima não é factível porque uma **Column** se expande para preencher todo o espaço disponível e um **ListView** faz o mesmo. Ambos não têm uma altura definida.

Você receberia um **erro** porque ambos tentam expandir para cobrir todo o espaço, mas não há nenhum widget pai com uma altura fixa.

```
Column(
    children: [
        const Text("My name"),
        const Text("My surname"),

        ListView(
            children: const [
                Text("Skill 1"), Text("Skill 2"),
            ]
        )
    ]
);
```

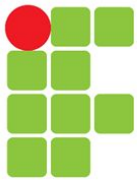


Scrolling e Constraints

Solução: A solução é sempre ter certeza de que eles estão dentro de um widget que define uma **altura finita.** Existem algumas maneiras de fazer isso:

- Usar o widget **Expanded**
- Atributo **shrinWrap: true** do widget **ListView**
- Utilizar as classes **Container** e **SizedBox.**

Scrolling e Constraints

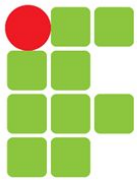


Usar o widget Expanded

```
Column(
    children: [
      const Text("My name"),
      const Text("My surname"),
      Expanded(
        child: ListView(children: const [
          Text("Skill 1"),
          Text("Skill 2"),
        ]),
      ),
    ],
  ),
```

Um widget que expande um filho de uma **Row**, **Column** para que o widget *child* preencha o espaço disponível.

Scrolling e Constraints



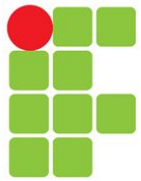
Atributo shrinWrap: true do widget ListView

```
Column(
    children: [
      const Text("My name"),
      const Text("My surname"),
      ListView(
        shrinkWrap: true,
        children: const [
          Text("Skill 1"),
          Text("Skill 2"),
        ],
      ),
    ],
  ),
```

shrinkWrap reduz a altura do **ListView** para o tamanho do conteúdo.

É significativamente mais caro computacionalmente do que expandir para o tamanho máximo permitido pois o conteúdo pode expandir e contrair durante a rolagem, o que significa que a visualização precisa ser recalculada.

Scrolling e Constraints



Atributo shrinWrap: true do widget ListView

SizedBox widget força o seu filho a ter uma largura e/ou altura específica.

```
Column(
            children: [
              Text("My name"),
              Text("My surname"),
              SizedBox(
                height: 200, // Necessário definir altura
                child: ListView(
                  children: [Text("Skill 1"), Text("Skill 2")],
                ),
              )
            ],
          ),
```

Scrolling e Cons

Atributo shrinWrap: tru

```
Column(
          children: [
            Text("My name
            Text("My surn
            Container(
              height: 200
              child: List
                children:
              ),
            )
          ],
        ),
```

Container widget também força o seu filho a ter uma largura e/ou altura específica.

Na prática, Container une três tipos de widget dentro de si.

- De **painting**: Para colorização como o *widget* **DecoratedBox**.
- De **positioning**: Para posicionamento como faz o widget **Padding, Positioned**.
- De ***sizing***: como faz **SizedBox, ConstrainedBox, FittedBox** e **RotatedBox**.

Widgets Comuns de Layout

O Flutter tem uma rica biblioteca de widgets de layout. Aqui estão alguns dos mais comumente usados.

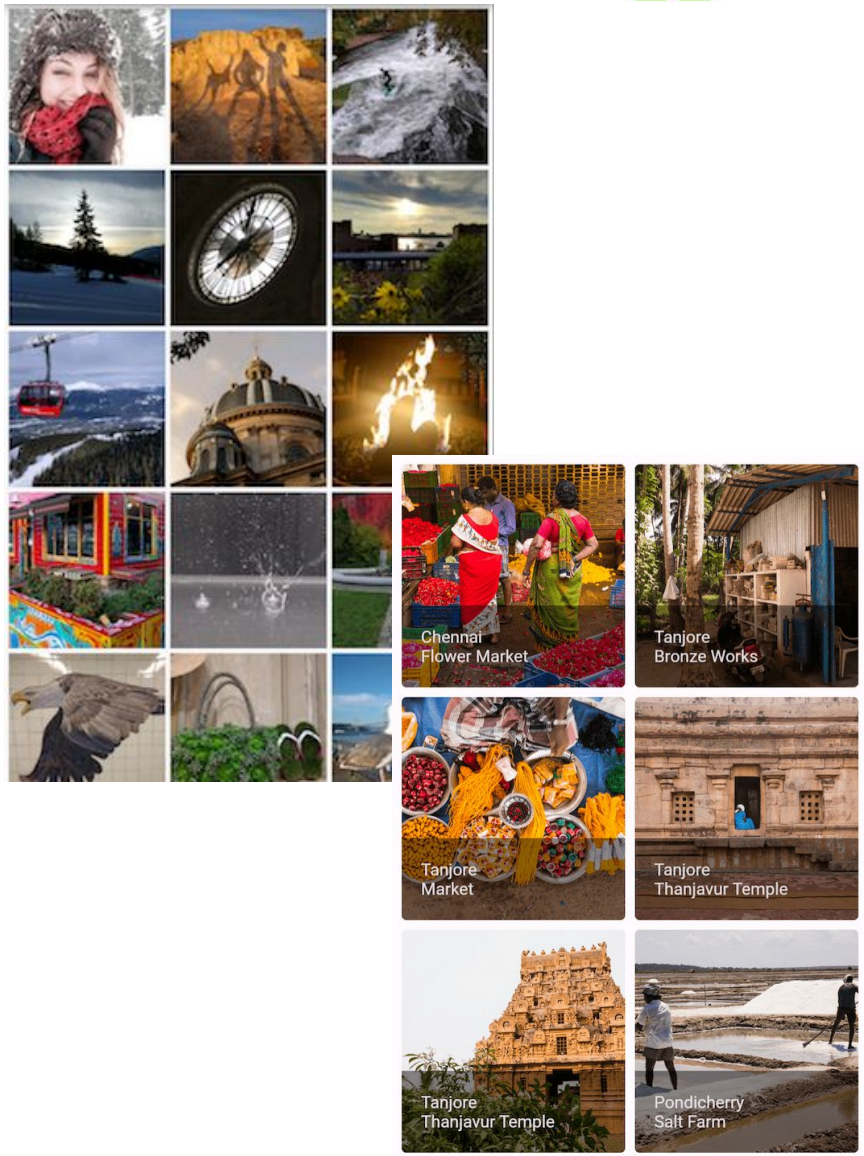
- **Container**: Adiciona preenchimento, margens, bordas, cor de fundo ou outras decorações a um widget. (Já conhecido!)
- **GridView**: Dispõe os widgets como uma grade rolável. (Novidade!)
- **ListView**: Dispõe os widgets como uma lista rolável. (Já conhecido!)
- **Stack**: Sobrepõe um widget sobre outro. (Já conhecido!)
- **Card**: Organiza informações relacionadas em uma caixa com cantos arredondados e sombra projetada. (Novidade!)

Widgets Comuns de Layout

GridView

Use **GridView** para dispor widgets como uma lista bidimensional. O **GridView** fornece duas listas pré-fabricadas, ou você pode construir sua própria grade personalizada.

Quando um **GridView** detecta que seu conteúdo é muito longo para caber na caixa de renderização, ele adiciona rolagem de tela automaticamente.



Widgets Comuns de Layout

GridView

Existem muitos construtores para **GridView**:

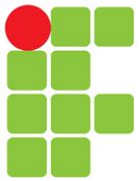
- **GridView(child):** define-se um grid fixo, com itens já existentes na sua construção.
- **GridView.count(crossAxisCount):** define-se apenas o número de colunas.
- **GridView.builder(itemBuilder):** para listas grandes ou de quantidade de itens não definidos. Otimiza uso de memória
- **GridView.extent(children):** define um tamanho máximo dos elementos, ao invés de definir o número de colunas.

Widgets Comuns de Layout

GridView(child): define-se um grid fixo, com itens já existentes na sua construção.

```
GridView(
  gridDelegate: SliverGridDelegateWithFixedCrossAxisCount(
    crossAxisCount: 2,
  ),
  children: [
    Container(color: Colors.red),
    Container(color: Colors.blue),
    Container(color: Colors.green),
    Container(color: Colors.yellow),
  ],
),
```

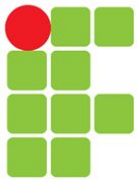
Widgets Comuns de Layout



GridView.count(crossAxisCount): define-se apenas o número de colunas.

```
GridView.count(
  crossAxisCount: 2, // Define o número de colunas
  children: List.generate(6,
      (index) => Container(color:
          Colors.blue[int.parse("${index}00")])),
    )
  )
),
```

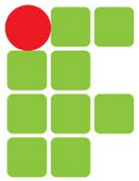
Widgets Comuns de Layout



GridView.builder(itemBuilder): para listas grandes ou de quantidade de itens não definidos. Otimiza uso de memória. Lógica de implementação semelhante ao do **ListView.builder.**

```
GridView.builder(
  gridDelegate: SliverGridDelegateWithFixedCrossAxisCount(
    crossAxisCount: 2,
  ),
  itemCount: 20,
  itemBuilder: (context, index) {
    return Image.network("https://picsum.photos/200");
  },
),
```

Widgets Comuns de Layout



GridView.extent(children): define um tamanho máximo dos elementos, ao invés de definir o número de colunas.

```
GridView.extent(
  maxCrossAxisExtent: 150, // Largura máxima de cada item
  children: List.generate(10,
      (index) =>
        Container(color:
              Colors.purple[int.parse("${index}00")]
        )
   ),
),
```

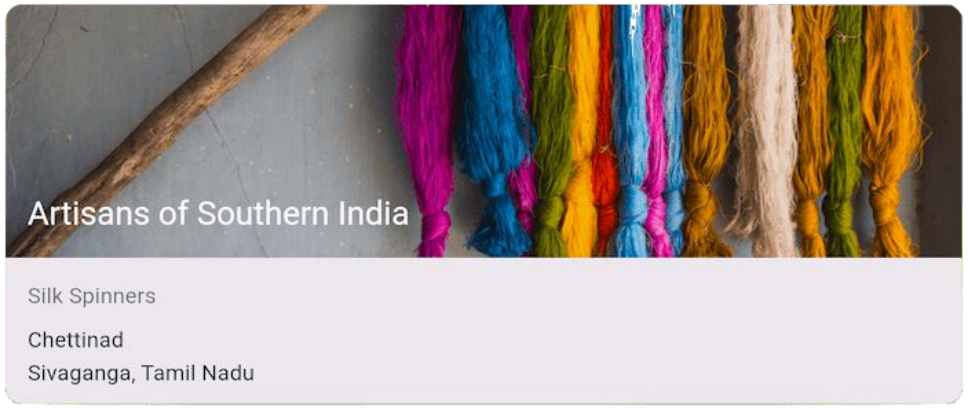
Widgets Comuns de Layout

Card (Específico para o Material)

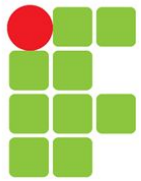
Contém porções de informação relacionadas e pode ser composto de quase qualquer widget, mas é frequentemente usado com **ListTile**.

Card tem um único filho, mas seu filho pode ser uma **Column, Row, ListView, GridView** ou outro widget que suporte vários filhos.

Por padrão, um **Card** reduz seu tamanho para 0 por 0 pixels. Você pode usar **SizedBox** para restringir o tamanho de um card.

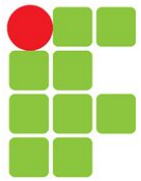


Widgets Comuns de Layout



```
Widget _buildCard() {
  return SizedBox(
    height: 210,
    child: Card(
      child: Column(
        children: [
          // aqui estarão os ListTiles
        ],
      ),
    ),
  );
}
```

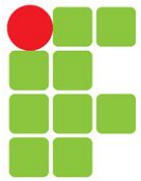
Widgets Comuns de Layout



```
Widget _buildCard() {
  return SizedBox(
    height: 210,
    child: Card(
      child: Column(
        children: [
          // aqui estarão os ListTiles
        ],
      ),
    ),
  );
}
```

Dica
Para minimizar a confusão visual que pode resultar de um código de layout muito aninhado, implemente partes da interface do usuário em variáveis e funções.

Widgets Comuns de Layout



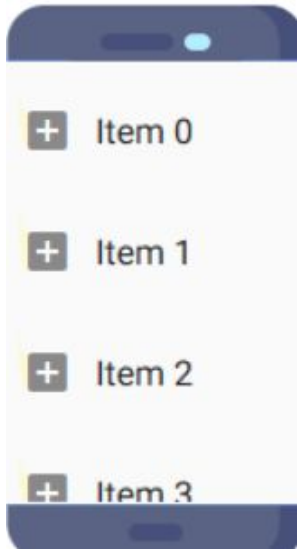
```
ListTile(
  title: const Text(
    '1625 Main Street',
    style: TextStyle(fontWeight: FontWeight.w500),
  ),
  subtitle: const Text('My City, CA 99984'),
  leading: Icon(Icons.restaurant_menu, color: Colors.blue[500]),
),
const Divider(), // Linha divisória
ListTile(
  title: const Text(
    '(408) 555-1212',
    style: TextStyle(fontWeight: FontWeight.w500),
  ),
  leading: Icon(Icons.contact_phone, color: Colors.blue[500]),
),
ListTile(
  title: const Text('costa@example.com'),
  leading: Icon(Icons.contact_mail, color: Colors.blue[500]),
),
```

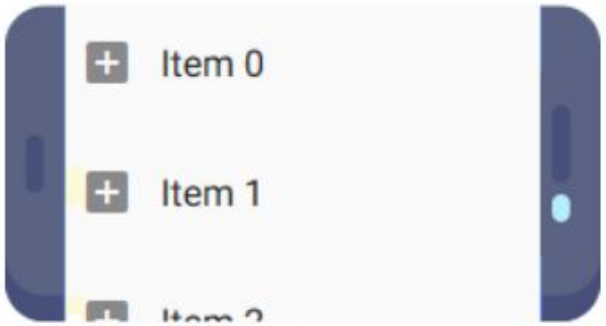
Responsividade

Aplicativos de alta qualidade são responsivos porque se adaptam automaticamente ao tamanho da tela, reorganizando a IU para preencher adequadamente todo o espaço disponível.

Cenário: Uso do *ListView* em diferentes orientações.

ListView com uma série de itens na posição vertical de tela.





ListView com os mesmo itens na posição horizontal de tela. Note os espaços em branco, além de conter menos itens na tela.

Responsividade

Aplica
autom
preen

Cenár

ListView
série d
posição
de tela.

De modo geral, se você planeja executar seu aplicativo Flutter em dispositivos móveis, desktop e web haverá enormes diferenças no tamanho da tela.

Você realmente precisa levar isso em consideração e criar a UI por consequência do dispositivo que ela está sendo apresentada.

Para tal, podemos utilizar o **LayoutBuilder**.

ptam
para

stView com os
esmo itens na
osição horizontal
e tela. Note os
paços em
anco, além de
nter menos
ens na tela.

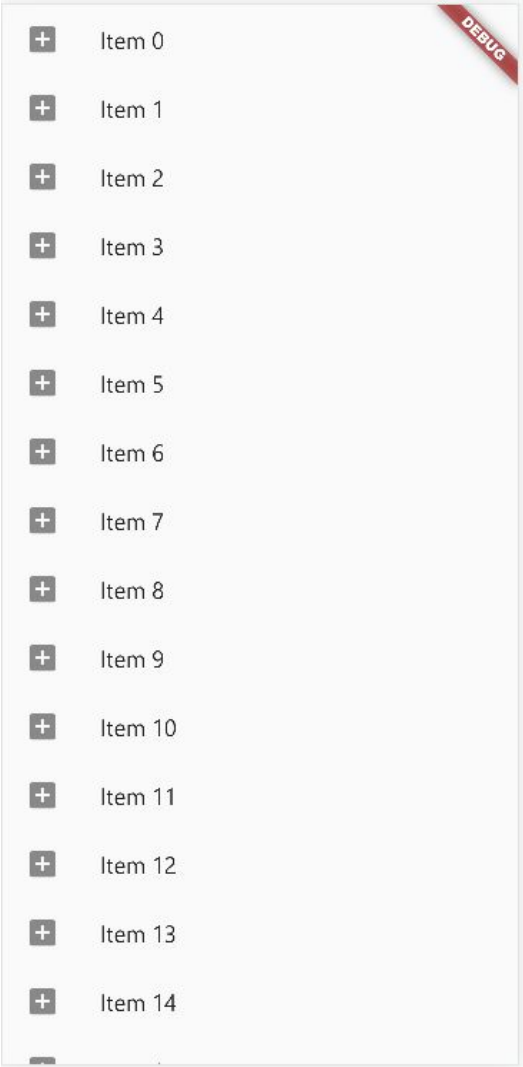
Responsividade: LayoutBuilder

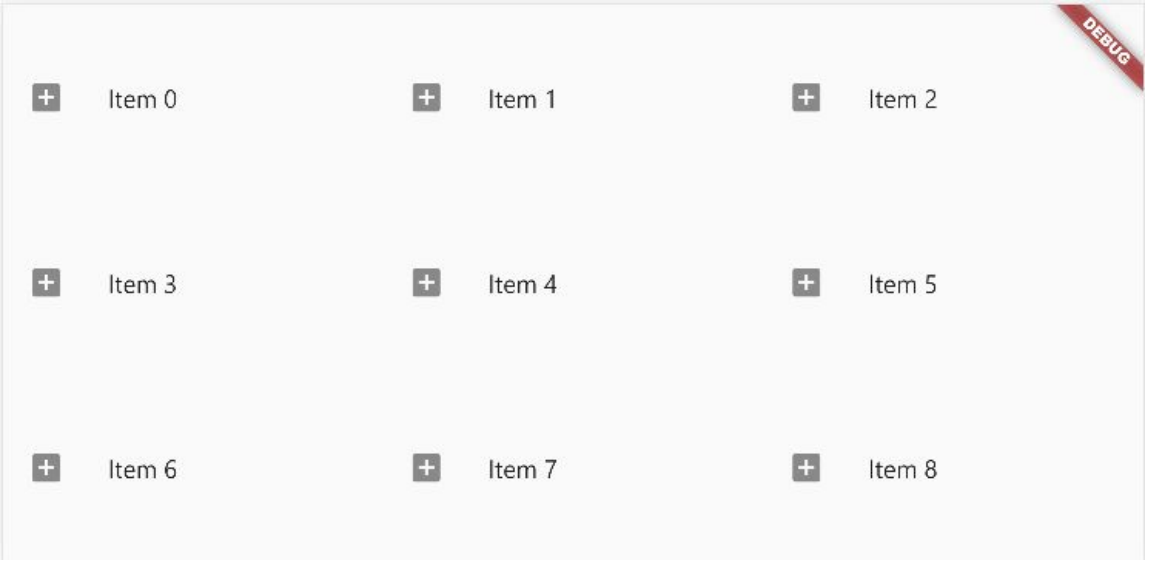
O Widget *LayoutBuilder* fornece informações sobre as restrições do pai, como o largura e a altura.

Considere realmente usar esta classe para tornar seus aplicativos responsivos porque ela pode ser usado para decidir como organizar a UI de acordo com o espaço disponível

```
LayoutBuilder(
    // dimensões do Widget pai são obtidas pelo parâmetro "sizes"
    builder: (BuildContext context, BoxConstraints sizes) {
      // Lógica de responsividade
    },
)
```

Exemplo LayoutBuilder





ListView se comprimento menor que 500px .

GridView se comprimento maior que 500px .

Exemplo LayoutBuilder

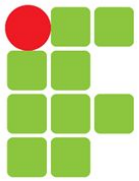
widget/list_view.dart

```
class ListData extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return ListView.builder(
        itemCount: 100,
        itemBuilder: (context, id) {
          return ListTile(
            leading: Icon(Icons.add_box),
            title: Text("Item $id"),
          );
        });
  }
}
```

widget/grid_view.dart

```
class GridData extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    var size = MediaQuery.of(context).size;
    final double itemHeight = size.height / 2;
    final double itemWidth = size.width / 2;
    return GridView.count(
      crossAxisCount: 3,
      childAspectRatio: (itemWidth / itemHeight),
      children: List.generate(
        100,
        (index) {
          return Center(
            child: ListTile(
              leading: Icon(Icons.add_box),
              title: Text("Item $index"),
            ),
          );
        },
      ),
    );
  }
}
```

Exemplo LayoutBuilder



main.dart

```
class MyApp extends StatelessWidget {
  // Este widget é a raiz da sua applicação.
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      // LayoutBuilder vai retornar o Widget dependendo
      // das medidas de seu Widget pai.
      body: LayoutBuilder(
        builder: (context, sizes) {
          if (sizes.maxWidth < 500) {
            return ListData();
          }
          return GridData();
        },
      ),
    );
  }
}
```

A decisão do que será renderizado está aqui.

Responsividade: MediaQuery

A classe *MediaQuery* é uma espécie de versão mais poderosa do *LayoutBuilder* porque está sempre disponível e oferece mais controle sobre várias configurações da tela.

Usá-la requer apenas um contexto:

```
MediaQuery.of(context)
```

Entretanto:

- ***MediaQuery*** retorna informações de dimensões da tela, que nunca muda em um dispositivo (pode mudar com a rotação da tela);
- ***LayoutBuilder*** trás informações de dimensões do **Widget pai**, que pode ser menor do que a da tela, podendo mudar e leva em consideração as dimensões de outros Widgets.

Responsividade: MediaQuery

Algumas informações de tela que podemos obter com MediaQuery.

Tamanho da tela:

```
MediaQuery.of(context).size.width; //comprimento
```

```
MediaQuery.of(context).size.height; //altura
```

Orientação:

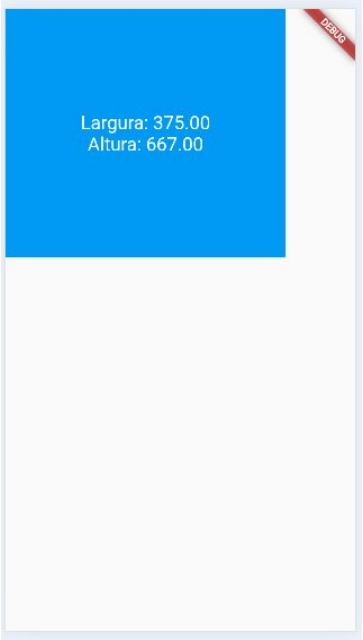
```
MediaQuery.of(context).orientation;
```

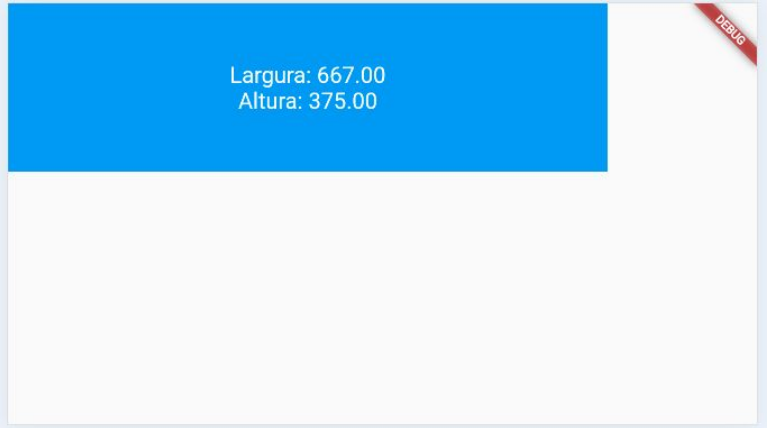
Retorna um valor do enum Orientation:

```
Orientation.landscape;
Orientation.portrait;
```

Responsividade: MediaQuery

```
class LayoutWidget extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    // Obtém as dimensões da tela
    double larguraTela = MediaQuery.of(context).size.width;
    double alturaTela = MediaQuery.of(context).size.height;

    return Container(
      width: larguraTela * 0.8, // 80% da largura da tela
      height: alturaTela * 0.4, // 40% da altura da tela
      color: Colors.blue,
      alignment: Alignment.center,
      child: Text(
        'Largura: ${larguraTela.toStringAsFixed(2)}\nAltura:
${alturaTela.toStringAsFixed(2)}',
        textAlign: TextAlign.center,
        style: TextStyle(color: Colors.white, fontSize: 20),
      ),
    );
  }
}
```



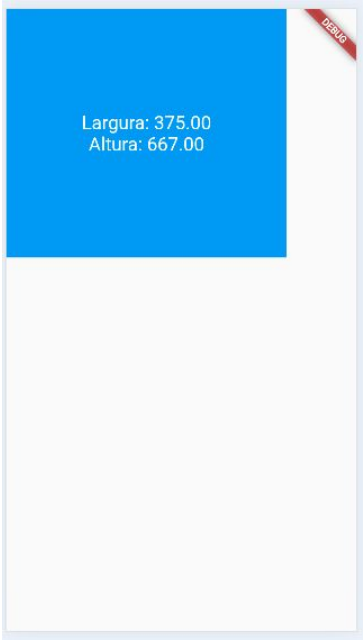


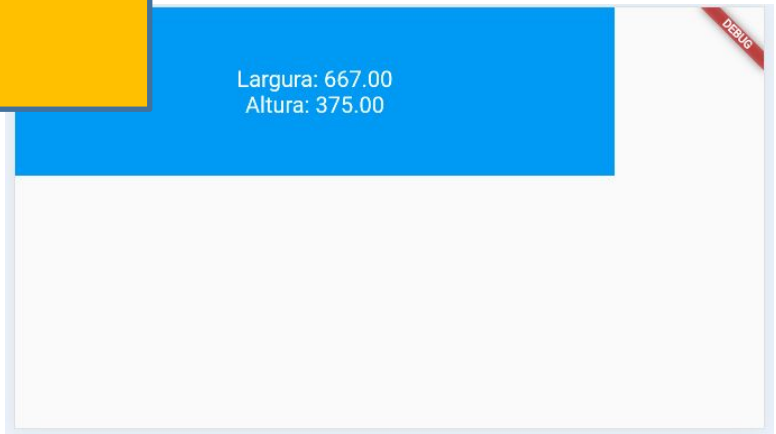
Responsividade: MediaQuery

```
class LayoutWidget extends StatelessWidget {
  @override
  Widget build(Buil
    // Obtém as dim
    double larguraT
    double alturaTe

    return Containe
      width: largur
      height: altur
      color: Colors
      alignment: Al
      child: Text(
        'Largura: $
${alturaTela.toStringAsFixed(2)}',
        textAlign: TextAlign.center,
        style: TextStyle(color: Colors.white, fontSize: 20),
      ),
    );
  }
}
```

Vejam outro exemplo no link:

https://www.youtube.com/watch?v=Mi7dizGicJo





Widgets de Inputs

Em termos básicos, podemos definir os widgets em quatro grupos principais.

- Layout
- **Input**
- **Dialogs, Popups e Messages**
- Navegação

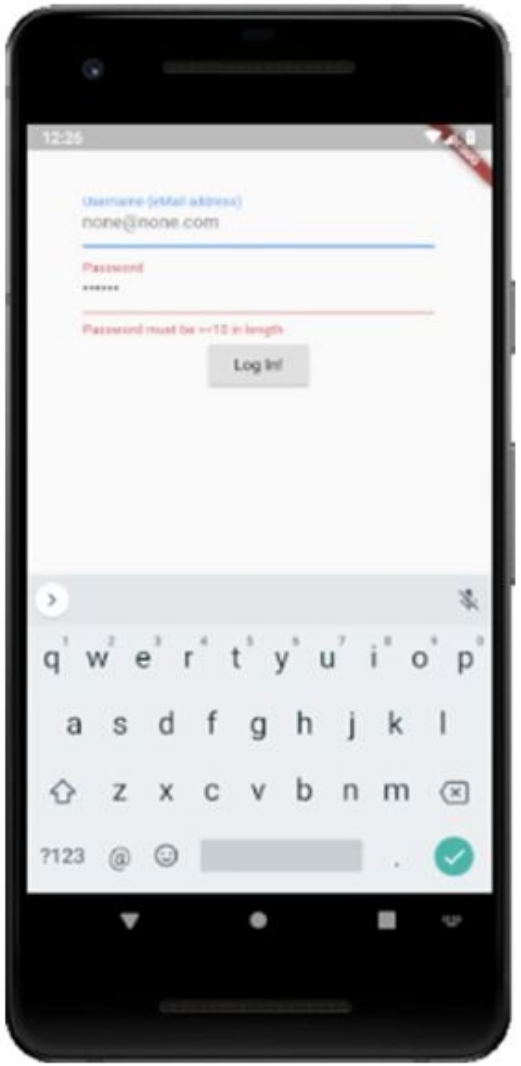
Widgets de inputs

Utilizados para se obter dados do usuário.

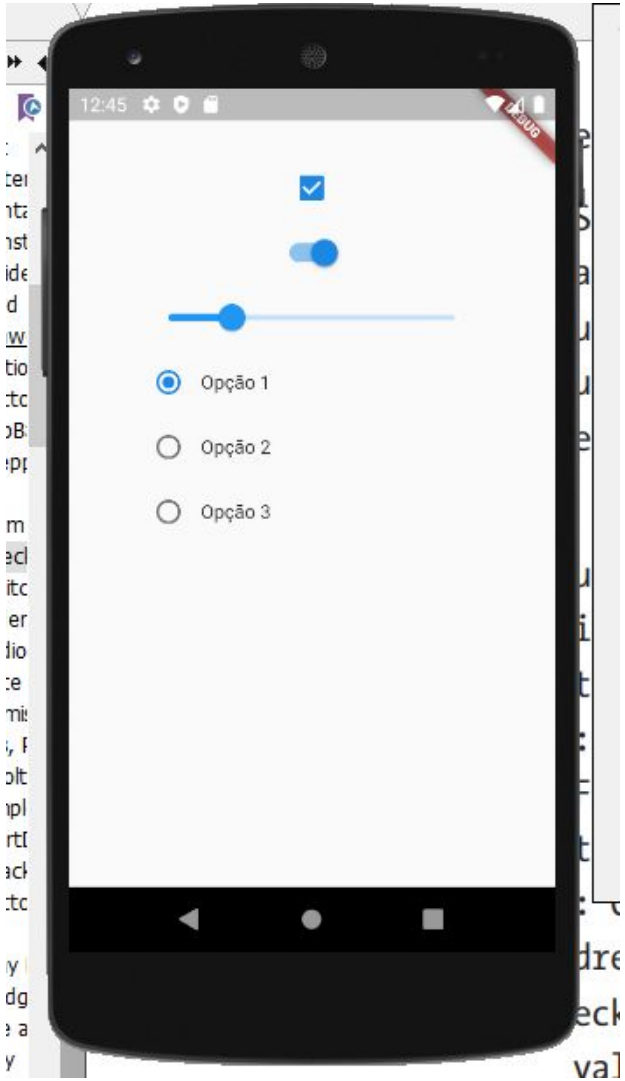
Exemplos:

- Checkbox
- Radio
- Switch
- Slider
- DropdownButton
- Date and Time Pickers
- Form

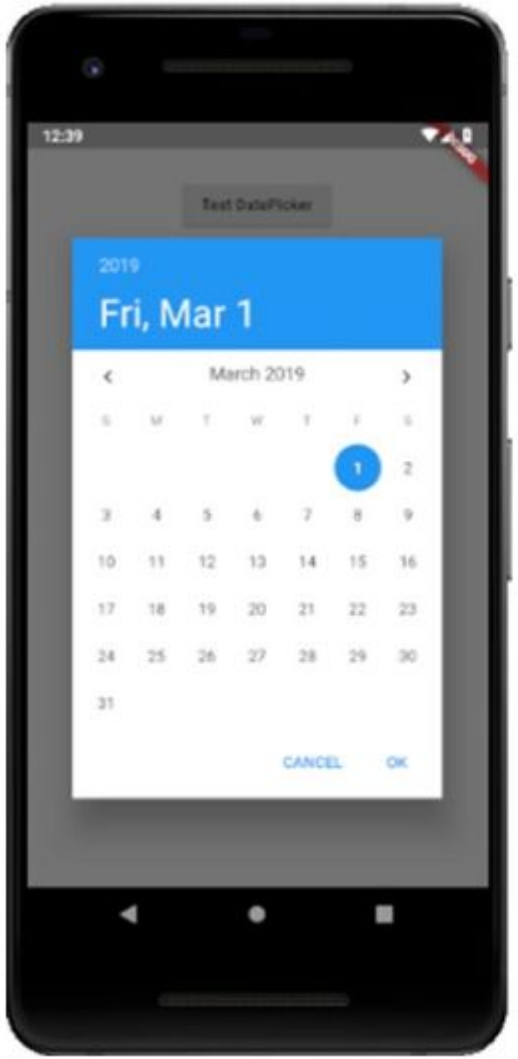
Widgets de Input



Form



Checkbox, Switch, Slider, Radio



Date and Time Pickers

ATENÇÃO!! ANTES DE COMEÇARMOS!

A maioria dos inputs que vamos verificar a seguir funcionam apenas dentro de um Widget com estado: O StatefulWidget. Copie essas duas classes em um único arquivo Dart.

```
class MyApp extends StatefulWidget{
  @override
  State<StatefulWidget> createState() {
    return MyAppState();
  }
}
```

```
class MyAppState extends State<MyApp> {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
        // Montagem da tela
    );
  }
}
```

ATENÇÃO!! ANTES DE COMEÇARMOS!

A maioria dos inputs que vamos verificar a seguir funcionam apenas dentro de um Widget com estado: O StatefulWidget. Copie essas duas classes em um único arquivo Dart.

```
class MyApp extends StatefulWidget{
  @override
```

```
class MyAppState extends State<MyApp> {
  @override
```

Sempre que precisarmos atualizar o estado do Widget iremos utilizar esta função.

```
setState(() {
    // Altera o estado (campos) do widget
});
```

```
}
```

Widgets de input

***Opcao* é um enum**

```
enum Opcoes{opcao1, opcao2}
```

RadioGroup e Radio	Um botão de opção do Material Design. Usado para selecionar entre vários valores mutuamente exclusivos dentro de um **RadioGroup**. Quando um botão de opção em um grupo é selecionado, os outros botões de opção no grupo deixam de ser selecionados.	

```
// Agrupa os Radios Buttons
RadioGroup<Opcoes>(
    groupValue: _opcao, // Estado Do Widget alterado pelos Radios.
    onChanged: (Opcoes? value) {
        setState(() {
        _opcao = value!; // _opcao é um atributo do Widget
        });
    },
    child: Column(
        children: [
            ListTile(
                title: const Text("Minha primeira opção"),
                // Radio é apenas o círculo que marca a opção
                leading: Radio(
                    // Valor da primeira opção. Vem do enum Opções. Deve ser um Radio por opção
                    value: Opcoes.opcao1,
                ),
            ),
            ListTile(
                title: const Text("Minha segunda opção"),
                // Radio é apenas o círculo que marca a opção
                leading: Radio(
                    // Valor da primeira opção. Vem do enum Opções. Deve ser um Radio por opção
                    value: Opcoes.opcao2,
                ),
            ),
        ],
    ),
),
```

Widgets de input

Checkbox	Uma caixa de seleção do Material Design. A caixa de seleção em si não mantém nenhum estado. Em vez disso, quando o estado da caixa de seleção muda, o widget chama o retorno de chamada onChanged.	`// ListTile para o Checkbox`
		`ListTile(`
		`  title: Text("Texto do Checkbox"),`
		`  leading: Checkbox(`
		`  // Vai definir se o checkbox está marcado ou não`
		`  value: _checado,`
		`  // EventHandler para alterar o state associado ao checkbox`
		`  onChanged: (value) { setState(() {`
		`  _checado = value;`
		`  });`
		`  },),`
		`),`

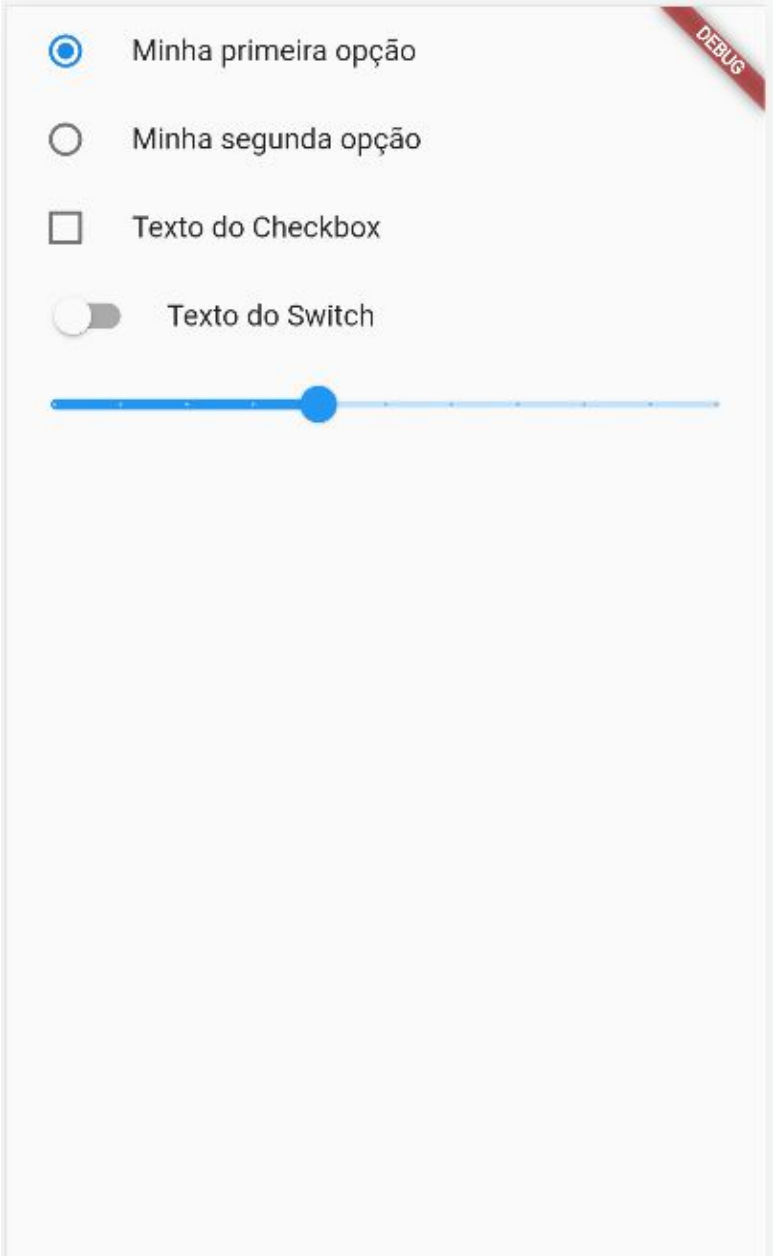
Switch	Uma opção de design de materiais. Usado para alternar o estado ligado/desligado de uma única configuração.	`// ListTile para o Checkbox`
		`ListTile(`
		`  title: Text("Texto do Switch"),`
		`  leading: Switch(`
		`  // Cor do Switch quando estiver ativado`
		`  activeColor: Colors.deepOrangeAccent[200],`
		`  // Vai definir se o switch está marcado ou não`
		`  value: _switchState,`
		`  // EventHandler para alterar o state associado ao switch`
		`  onChanged: alternarSwitch,`
		`  ),`
		`),`

Widgets de input

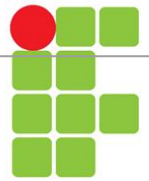
Slider	Um controle deslizante do Material Design. Usado para selecionar entre um intervalo de valores.	

```
ListTile(
        title: Text("valor: ${_sliderState.toStringAsPrecision(3)}"), //texto indicando o
valor do slider
           leading: SizedBox( // Necessário para definir um tamanho para o Slider
              width: 200,
              child: Slider(
                 max: 100, // Valor máximo do slider
                 divisions: 15, // Número de pontos em que o Slider será quabrado
                    value: _sliderState, // é um double
                    onChanged: (value) {
                      setState(() {
                        _sliderState = value;
         });
       },
     ),
  ),
),
```

Widgets de input



Widgets de input



DropdownButton	Um botão do Material Design para seleção em uma lista de itens. Um botão suspenso permite que o usuário selecione vários itens. O botão mostra o item atualmente selecionado, bem como uma seta que abre um menu para selecionar outro item.	

```
ListTile(
  title: Text("Selecione uma opção!"),
  leading: // <T> Especifica o tipo de objeto tratado pelo Dropdown
      DropdownButton<String>(
    // Variável que será atualizada ao selecionar item
    value: _dropdownValue,
    // A lista é mapeada para uma lista de DropdownMenuItem<String>
    items:
        _dropdownList.map<DropdownMenuItem<String>>(
          (item) {
            return DropdownMenuItem(
              value: item,
              child: Text(item!),
            );
          },
        ).toList(), // map retorna um Iterable, mas DropdownButton.items recebe uma
List.
    onChanged: (value) {
      setState(() {
        _dropdownValue = value as String;
      });
    }, // EventHandler ao mudar o dropdown
  ),
),
```

Widgets de input

showDatePicker()	Mostra uma caixa de diálogo contendo um seletor de data do Material Design. O Future retornado é resolvido para a data selecionada pelo usuário quando o usuário confirma a caixa de diálogo. Se o usuário cancelar o diálogo, será retornado nulo.	

```
// TimePicker é um modal, que é inflada de fora da árvore do Widget
  // Sendo assim, coletar uma hora é uma operação assíncrona,
  // por isso o Future<void>
  // BuildContext é necessário para o showTimePicker
  Future<void> _selectHour(BuildContext context) async {
    // Mostra o DatePicker e espera por uma data selecionada
    final TimeOfDay? picked = await showTimePicker(
        context: context,
        initialTime: TimeOfDay.now(),
        );

    // Se uma data for selecionada, atualiza o estado
    if (picked != null && picked != _horaSelecionada) {
      setState(() {
        _horaSelecionada = picked;
      });
    }
```

Sempre que utilizarmos **showTimePicker** temos que utilizá-los numa função assíncrona.

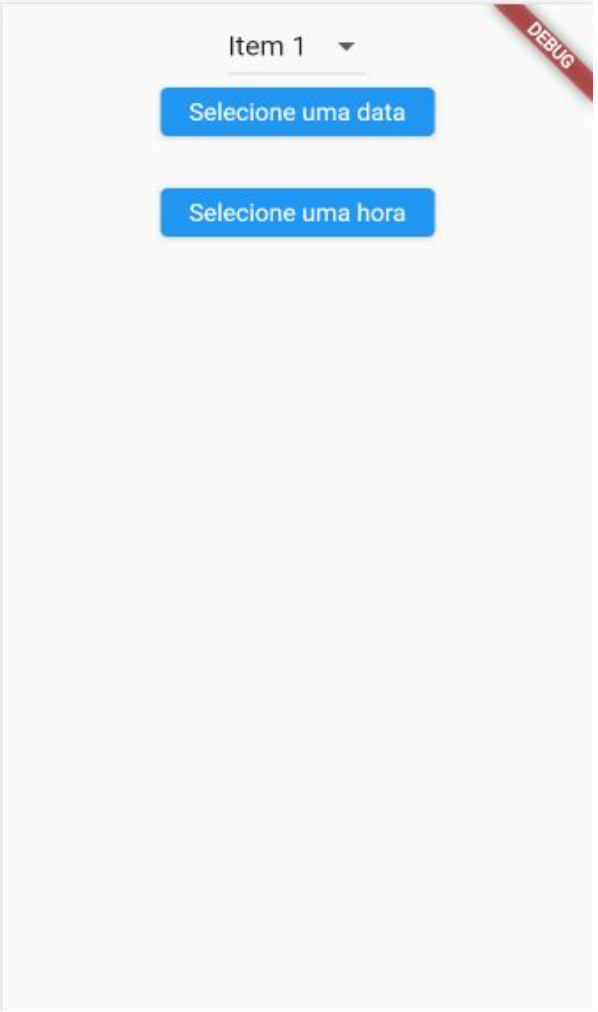
Widgets de input

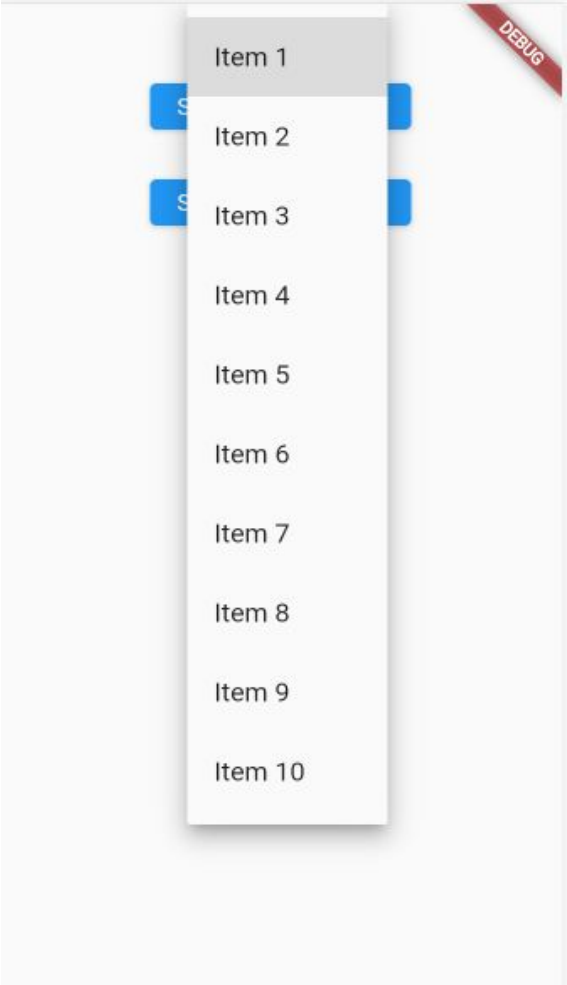
showTimePicker()	Mostra uma caixa de diálogo contendo um seletor de tempo do Material Design. O Future retornado é resolvido para o horário selecionado pelo usuário quando ele fecha a caixa de diálogo. Se o usuário cancelar o diálogo, será retornado nulo.	

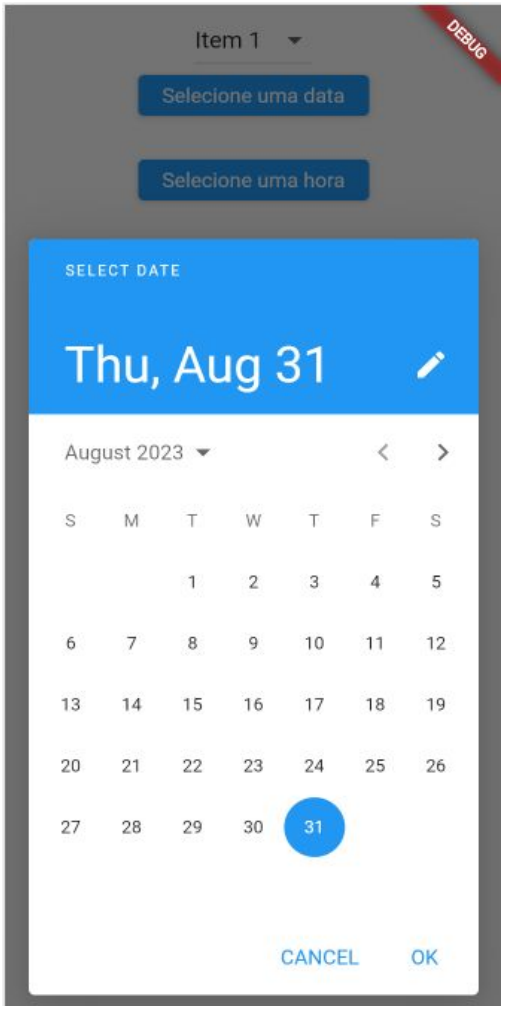
```
// DatePicker é um modal, que é inflada de fora da árvore do Widget
  // Sendo assim, coletar uma data é uma operação assíncrona,
  // por isso o Future<void>
  Future<void> _selectDate(BuildContext context) async {
    // Mostra o DatePicker e espera por uma data selecionada
    final DateTime? picked = await showDatePicker(
      context: context,
      initialDate: dataSelecionada,
      firstDate: DateTime(2023, 8),
      lastDate: DateTime(2101),
    );

    // Se uma data for selecionada, atualiza o estado
    if (picked != null && picked != dataSelecionada) {
      setState(() {
        dataSelecionada = picked;
      });
    }
  }
```

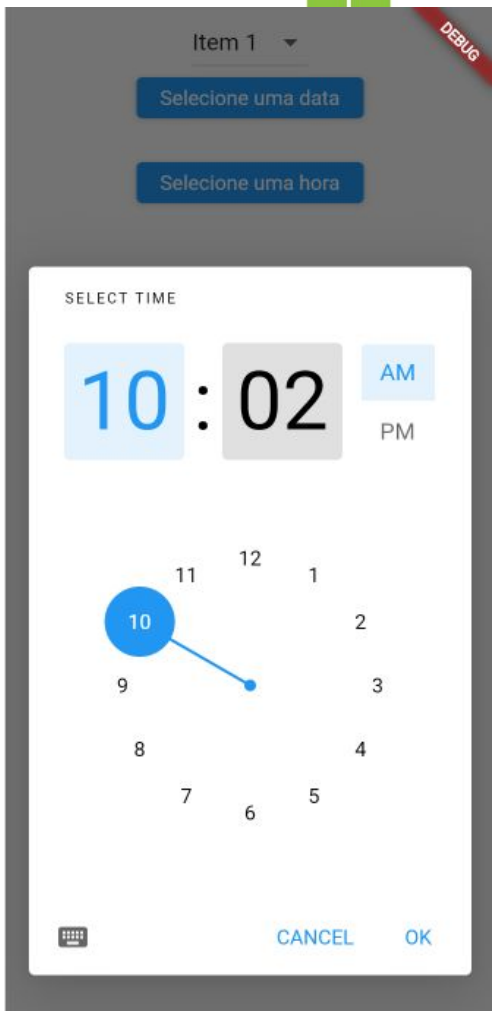
Sempre que utilizarmos **showDatePicker** temos que utilizá-los numa função assíncrona.

Widgets de input

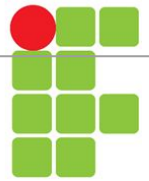








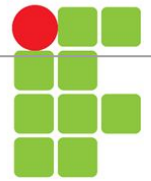
Widgets de input



Form	Um contêiner opcional para agrupar vários widgets de campo de formulário (por exemplo, widgets TextField).	

```
return Form(
  key: _formKey, //a key é importante para referenciar o Form posteriormente
  child: Column(
    children: <Widget>[
      TextFormField(
       decoration: const InputDecoration(
         hintText: 'Entre com seu email',
       ),
       validator: (String? value) { // faz a validação de um campo do formulário
         if (value == null || value.isEmpty)
           return 'Por favor entre com algum texto';
         return null;
       },
      ),
      Padding(
       padding: const EdgeInsets.symmetric(vertical: 16.0),
       child: ElevatedButton( // ativa a validação e a ação junto aos dados do Form
         onPressed: () {
           //
           if (_formKey.currentState!.validate()) {
             // Processa os dados
           }
         },
         child: const Text('Enviar'),
       ),
      ),
    ],
  ),
```

Widgets de input



Form

Um contêiner opcional para agrupar vários widgets de campo de formulário (por exemplo, widgets TextField).

```
// campo do Widget
GlobalKey<FormState> _formKey =
GlobalKey<FormState>();
```

```
return Form(
  key: _formKey, //a key é importante para referenciar o Form posteriormente
  child: Column(
    children: <Widget>[
      TextFormField(
        decoration: const InputDecoration(
          hintText: 'Entre com seu email',
        ),
        validator: (String? value) { // faz a validação de um campo do formulário
          if (value == null || value.isEmpty)
            return 'Por favor entre com algum texto';
          return null;
        },
      ),
      Padding(
        padding: const EdgeInsets.symmetric(vertical: 16.0),
        child: ElevatedButton( // ativa a validação e a ação junto aos dados do Form
          onPressed: () {
            //
            if (_formKey.currentState!.validate()) {
              // Processa os dados
            }
          },
          child: const Text('Enviar'),
        ),
      ),
    ],
  ),
```

Widgets de input

Form	Um contêiner opcional para agrupar vários widgets de campo de formulário (por exemplo, widgets TextField).	

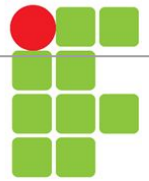
```
return Form(
  key: _formKey, //a key é importante para
  child: Column(
    children: <Widget>[
      TextFormField(
        decoration: const InputDecoration(
          hintText: 'Entre com seu email',
        ),
        validator: (String? value) { // faz a validação de um campo do formulário
          if (value == null || value.isEmpty)
            return 'Por favor entre com algum texto';
          return null;
        },
      ),
      Padding(
        padding: const EdgeInsets.symmetric(vertical: 16.0),
        child: ElevatedButton( // ativa a validação e a ação junto aos dados do Form
          onPressed: () {
            //
            if (_formKey.currentState!.validate()) {
              // Processa os dados
            }
          }
          child: const Text('Enviar'),
```

```
TextFormField(
  decoration: const InputDecoration(
    hintText: 'Entre com seu email',
  ),
  // Ativado por  formKey.currentState!.save()
  onSaved: (valorInput) {
    // Obtem o valor inserido pelo usuário
    print(valorInput);
  },
),
```

```
ElevatedButton(
  // ativa a validação e a ação junto aos dados do Form
  onPressed: () {
    _formKey.currentState!.save();
    if (_formKey.currentState!.validate()) {
      // Processa os dados
    }
  },
  child: const Text('Enviar'),
),
```

Podemos obter os dados do input acionado configurando uma função **onSave** no input e chamando **save()** do estado do Form.

Widgets de input



Form

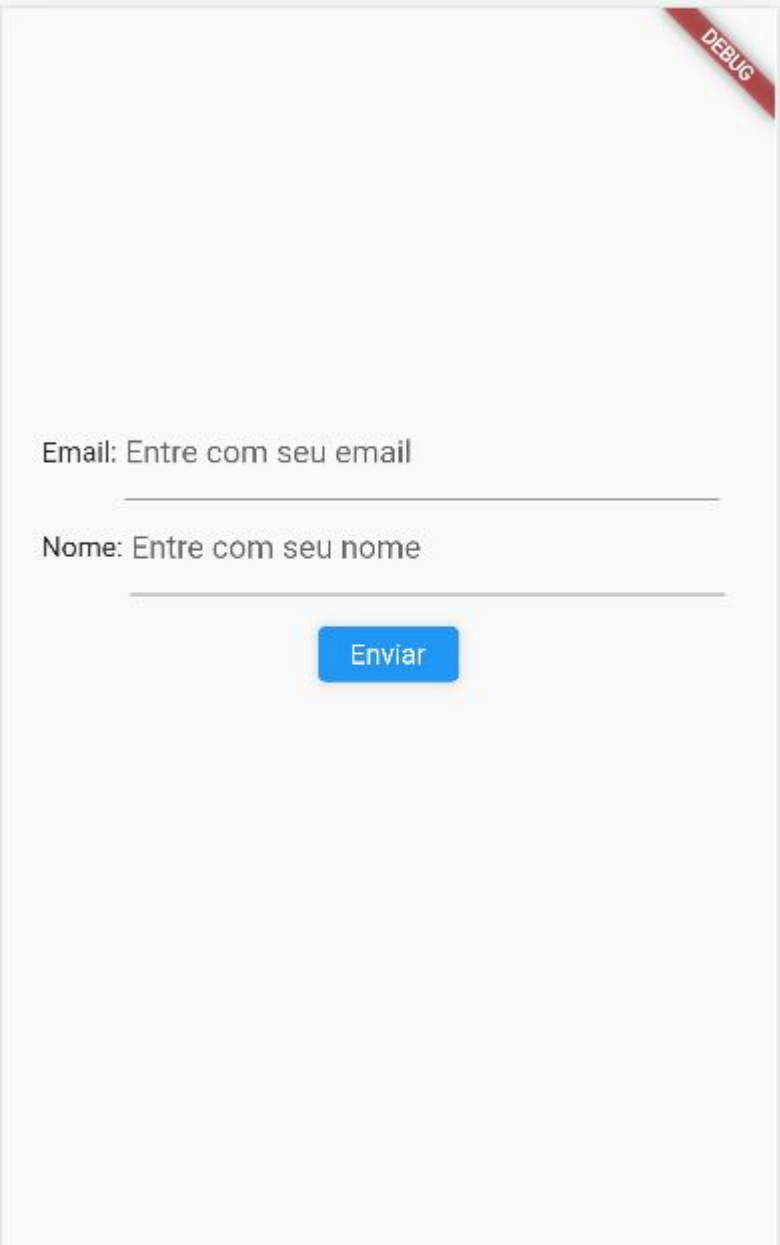
Um contêiner opcional para agrupar vários widgets de campo de formulário (por exemplo, widgets TextField).

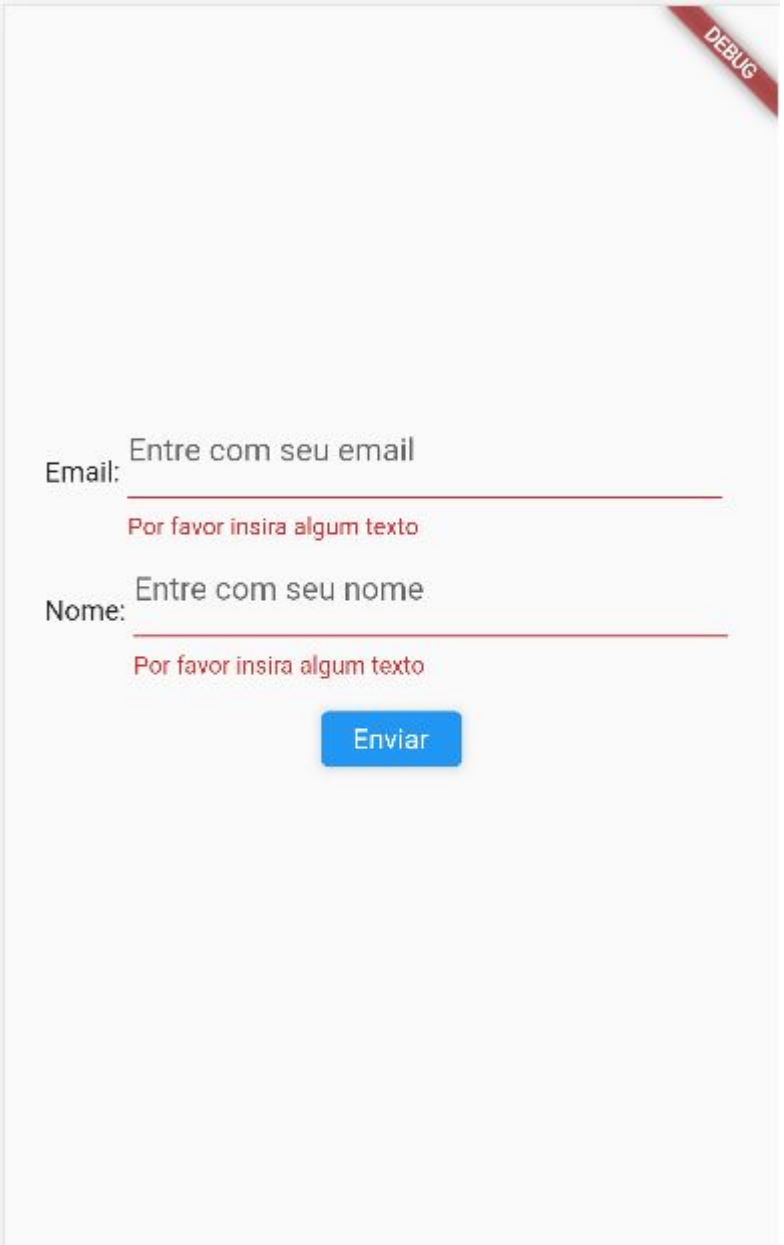
```
return Form(
  key: _formKey, //a key é importante para referenciar o Form posteriormente
  child: Column(
    children: <Widget>[
      TextFormField(
       decoration: const InputDecoration(
         hintText: 'Entre com seu email',
       ),
       validator: (String? value) { // faz a validação de um campo do formulário
         if (value == null || value.isEmpty)
           return 'Por favor entre com algum texto';
         return null;
       },
     ),
     Padding(
       padding: const EdgeInsets.symmetric(vertical: 16.0),
       child: ElevatedButton( // ativa a validação e a ação junto aos dados do Form
         onPressed: () {
           //
           if (_formKey.currentState!.validate()) {
             // Processa os dados
           }
         },
         child: const Text('Enviar'),
       ),
     ),
    ],
  ),
```

Você pode acessar o Form também pelo comando.

```
Form.of(context).validate();
```

Widgets de input



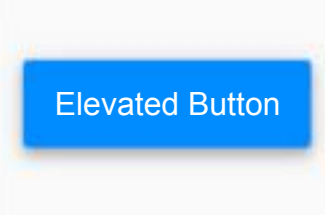


Widgets de Inputs

Buttons

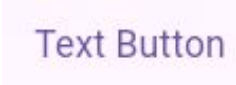
Flutter tem muitos widgets criados para seguir as diretrizes do Material Design para botões.

ElevatedButton



Este é um botão Android típico com uma forma retangular e uma elevação padrão (a sombra atrás). Quando pairado, a elevação aumenta para que você possa ver visualmente o interação com o botão.

TextButton



É muito semelhante a um *ElevatedButton* com a diferença de que aqui não há elevação.

OverflowBar

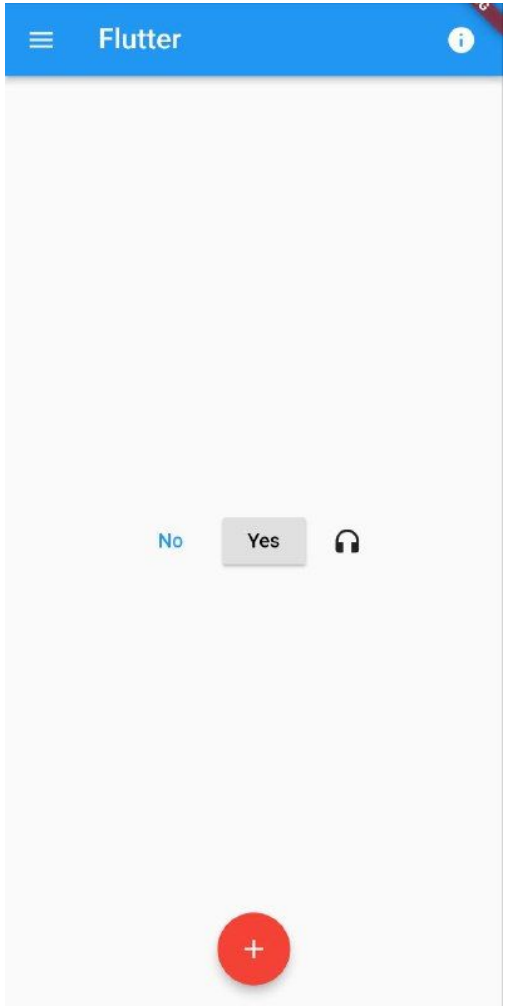
É um contêiner horizontal que contém uma série de botões. Pode ser útil nos casos em que você tem uma caixa de diálogo e deseja mostrar dois botões como NÃO e SIM.

IconButton

Você já viu um exemplo no AppBar como eles tornam os ícones clicáveis; é um botão cujo conteúdo é um ícone Material em vez de uma string simples.

Widgets de Inputs

Buttons




```
      body: Center(
          child: OverflowBar(
            alignment: MainAxisAlignment.center,
            children: [
              TextButton(
                onPressed: () {},
                child: const Text("No"),
              ),
              ElevatedButton(
                onPressed: () {},
                child: const Text("Yes"),
              ),
              IconButton(
                onPressed: () {},
                icon: Icon(Icons.headphones),
              ),
            ],
          ),
        ),
```

Widgets de diálogo

Widgets de Dialogs, Popups e Mensagens

Permitem interagir com o usuário, de mostrar-lhe algo, de certa forma "Fora do radar", o que significa que apresenta conteúdo que não faz parte diretamente da tela em que estão atualmente olhando no momento.

Exemplos:

- Tooltip
- SimpleDialog
- AlertDialog
- BottomSheet
- Snackbar (veremos depois)

Widgets de Dialogs, Popups e Mensagens

SimpleDialog

SnackBar

BottomSheet

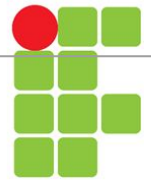
Widgets de Diálogo

Widgets de Dialogs, Popups e Mensagens

Esses widgets geralmente funcionam pela chamada de uma função que os **infla** acima da tela apresentada no momento, ou seja, ela funcionam como um **Modal**.

```
 // Temos que chamar a função para que ela saiba
como inflar a caixa de diálogo na tela
showDialog(
      context: context,
      builder: (BuildContext context) {
        // retorna o Wdiget de Dialog
    }
)
```

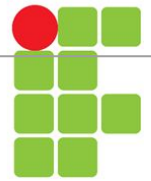
Widgets de diálogo



Tooltip	Uma dica de ferramenta do Material Design. As dicas de ferramentas fornecem rótulos de texto que ajudam a explicar a função de um botão ou outra ação da interface do usuário. Envolva o botão em um widget de dica de ferramenta e forneça uma mensagem que será mostrada quando o widget for pressionado por muito tempo.	`Tooltip(`
		`  message: 'Eu sou uma Tooltip',`
		`  child: Text('Passe o mouse sobre para mostrar o Tooltip.'),`
		`  // child: Text('Hover over the text to show a tooltip.'),`
		`),`

SimpleDialog	Uma caixa de diálogo simples do Material Design. Uma caixa de diálogo simples oferece ao usuário a escolha entre diversas opções. Uma caixa de diálogo simples possui um título opcional que é exibido acima das opções.	`var valor = await showDialog<String>(`
		`  context: context,`
		`  builder: (BuildContext context) {`
		`  return SimpleDialog(`
		`  title: const Text('Título do AlertDialog'),`
		`  children: [`
		`  // Representa uma função da caixa de SimpleDialog`
		`  SimpleDialogOption(`
		`  // EventHandler para quando esta opção for selecionada`
		`  onPressed: () {`
		`  // Volta o foco para a página abaixo do Dialog`
		`  Navigator.pop(context, Opcoes.opcao1.name.toString());`
		`  },`
		`  child: const Text('Opção 1'),`
		`  ),`
		`  ],`
		`  );`
		`  },`
		`  );`

Widgets de diálogo



AlertDialog	Uma caixa de diálogo de alerta do Material Design. Uma caixa de diálogo de alerta (também conhecida como caixa de diálogo básica) informa o usuário sobre situações que exigem confirmação. Uma caixa de diálogo de alerta possui um título opcional e uma lista opcional de ações.	

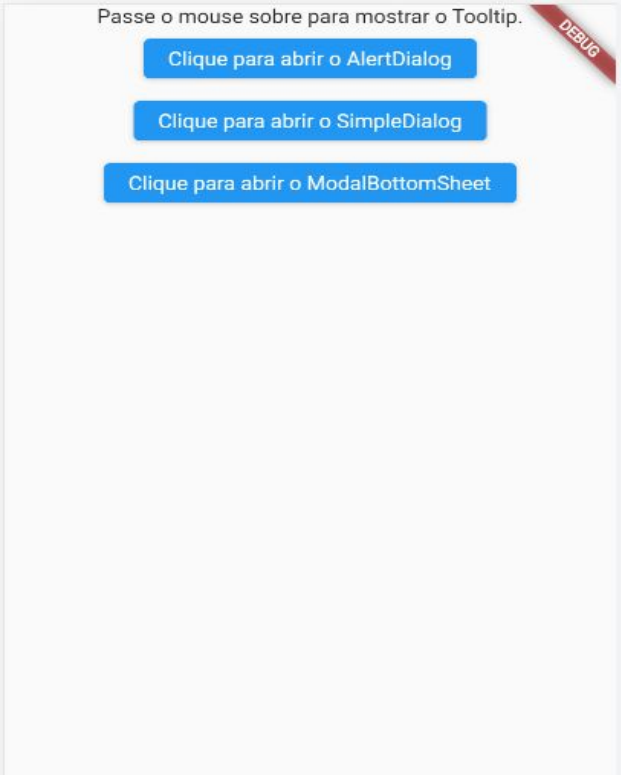
```
showDialog<String>(
      context: context,
      builder: (BuildContext context) {
        return AlertDialog(
          title: const Text('Título do AlertDialog'),
          // A documentação pede para utilizar este Widget
          content: SingleChildScrollView(
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: <Widget>[
                Text('Este é um AlertDialog.'),
                Text('Você gostaria de aprovar esta mensagem?'),
              ],
            ),
          ),
          actions: [
            TextButton(
              child: const Text('Aprovar'),
              onPressed: () {
                // Volta à página, fechando o AlertDialog
                Navigator.of(context).pop();
              },
            ),])
```

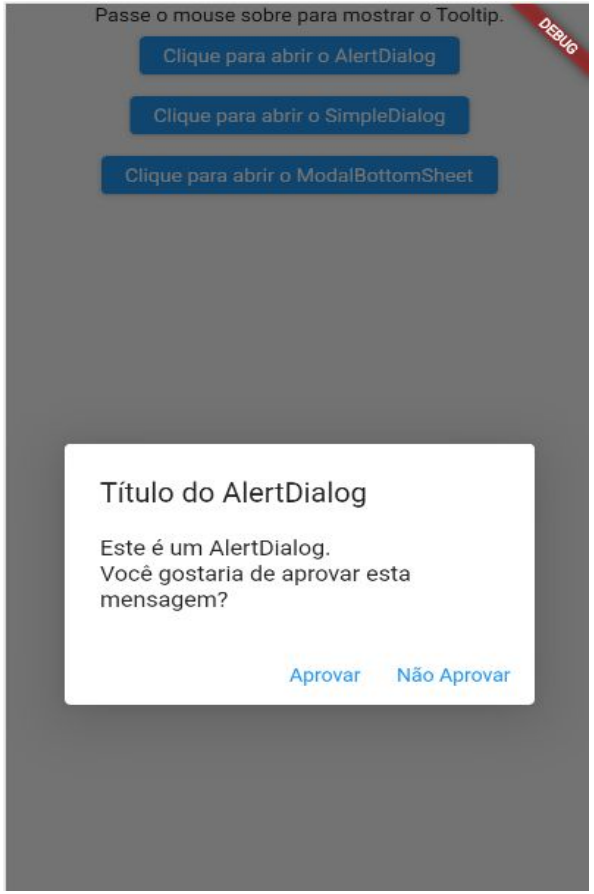
Widgets de diálogo

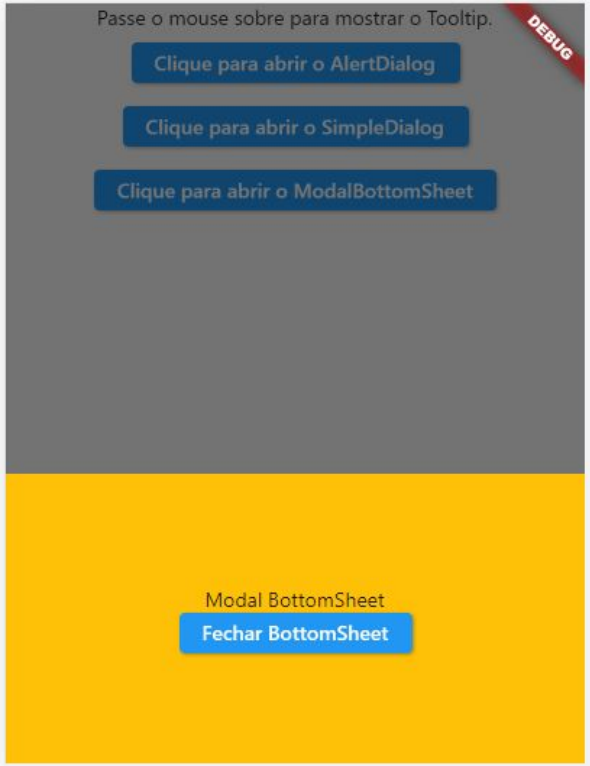
BottomSheet com **showModalBottomSheet()**	Mostra uma folha inferior modal do Material Design. Uma folha inferior modal é uma alternativa a um menu ou caixa de diálogo e evita que o usuário interaja com o restante do aplicativo.	

```
showModalBottomSheet<void>(
      context: context,
      builder: (BuildContext context) {
        return Container(
          height: 200,
          color: Colors.amber,
          child: Center(
            child: Column(
              mainAxisAlignment: MainAxisAlignment.center,
              mainAxisSize: MainAxisSize.min,
              children: <Widget>[
                const Text('Modal BottomSheet'),
                ElevatedButton(
                  child: const Text('Fechar BottomSheet'),
                  // Volta à tela que está embaixo do ModalBottomSheet
                  onPressed: () => Navigator.pop(context),
                ),
              ],
            ),
          ),
        );
```

Widgets de Dialogs, Popups e Mensagens

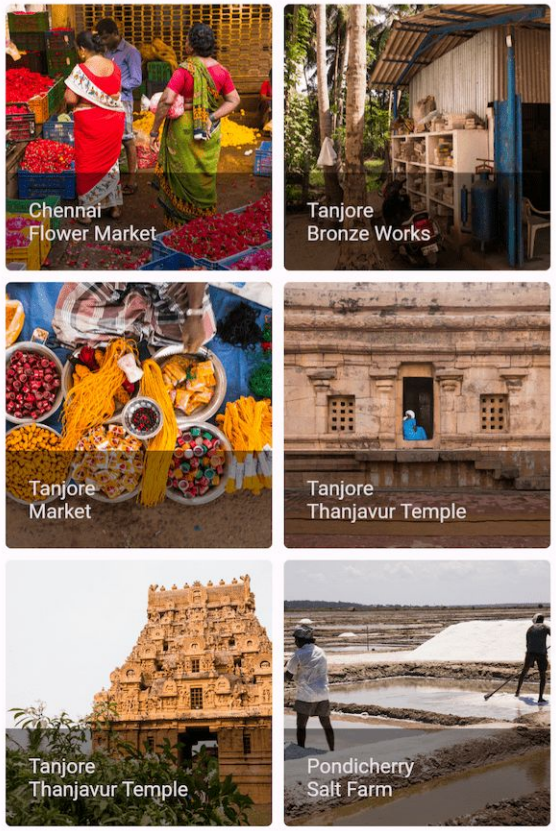






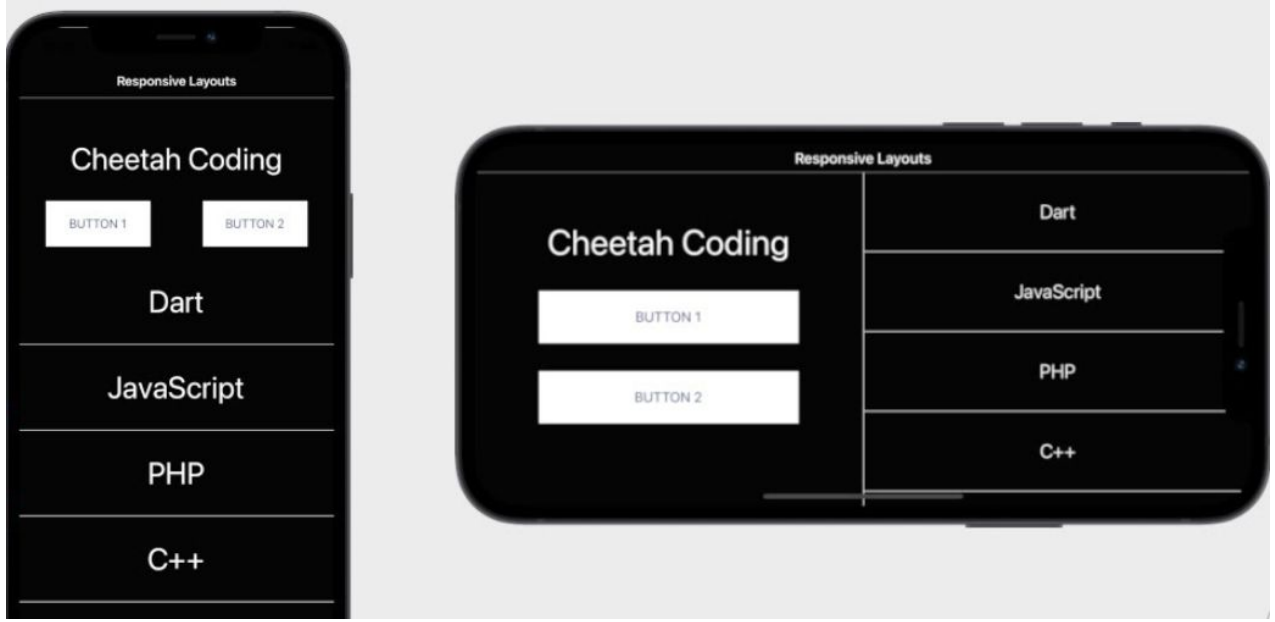
Exercícios

1. Implemente o **GridView** no layout da imagem abaixo. Utilize as imagens que desejar (baixando-as ou buscando na Internet com ***Image.network(url)***).



Exercícios

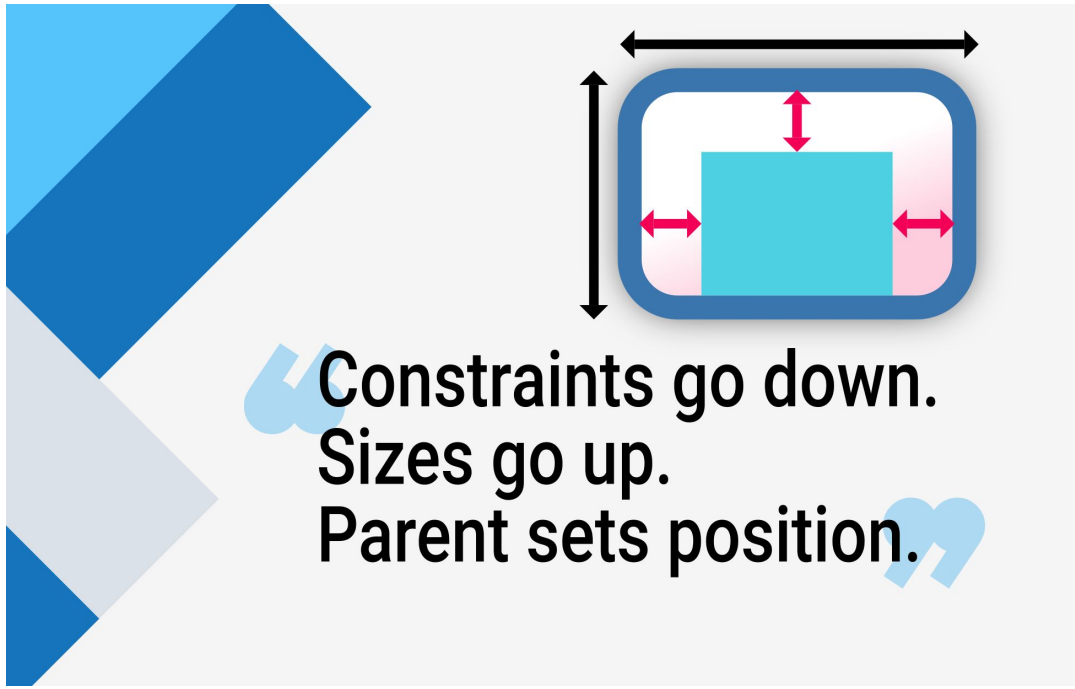
2. Desenvolva um app com base no layout a seguir aplicando responsividade.



Exercícios

3. Implemente em um projeto os códigos do link a seguir para compreender sobre restrições de layout.

https://docs.flutter.dev/ui/layout/constraints



Exercícios

4. Implemente um ***Form*** como na imagem abaixo. Busque mesclar ***Column*** com ***Row*** para obter o layout apresentado.

Mostre em um ***SnackBar*** os dados inseridos quando o botão *Submit* for pressionado.

Não sabe como capturar dados dos inputs? Veja como aqui https://docs.flutter.dev/cookbook/forms/retrieve-input

Exercícios

5. Você sabia que o ***FloatingActionButton*** (também conhecido como FAB) pode ser posicionado ao meio de um ***BottomBarNavigation***?

Aprenda como realizar essa configuração implementando o tutorial deste link.

https://codewithandrea.com/articles/bottom-bar-navigation-with-fab/

Verifique que ao selecionar o FAB, novas opções são apresentadas. Implemente também esse recurso.

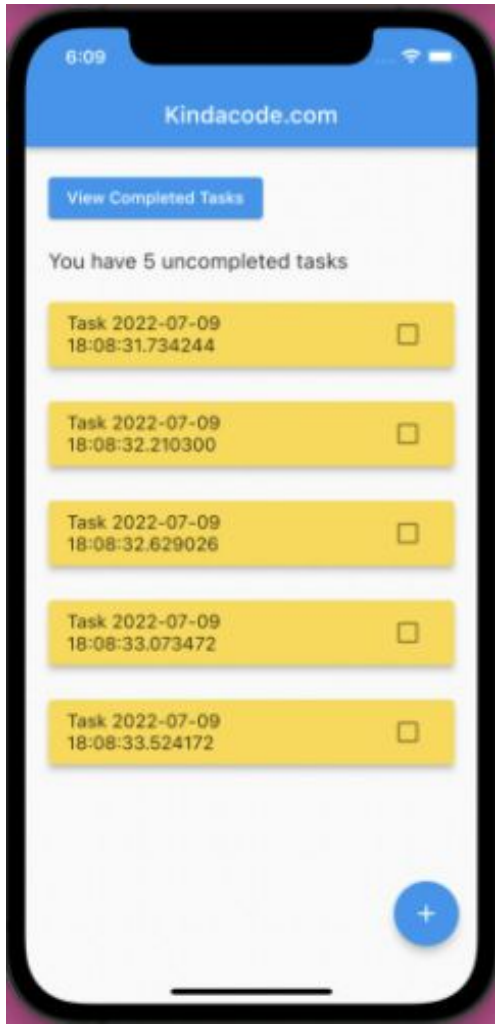
Exercícios

6. Crie a tela da imagem utilizando como raiz da árvore de widgets um ***MaterialApp e Scaffold***.

Para criar a lista de tarefas você deve utilizar também o ***ListView*** com o método builder para criar ***Cards*** para mostrar as tarefas. Utilize também ***CheckBox*** e ***ElevatedButton***. Ao clicar no ***FloatingActionButton***, faça aparecer um ***AlertDialog*** que apresente a mensagem "Você está no App de Notas de Tarefas".

Caso não conheça o Widget ***Card,*** verifique a documentação no link:

https://api.flutter.dev/flutter/material/Card-class.html



Referência

MIOLA, Alberto. **Flutter Complete Reference: Create Beautiful, Fast and Native Apps for Any Device.** In:___. Building UIs in Flutter. Independently published, 2020.

https://docs.flutter.dev/ui/layout

https://docs.flutter.dev/ui/widgets/material

https://api.flutter.dev/flutter/widgets/StatefulWidget-class.html